

Alegorytmy Ewolucyjne (AE)

Bartłomiej Wójcik

Numer indeksu: 327327

Spis treści

Implementacja Algorytmów Ewolucyjnych	2
AE1: podstawowy algorytm ewolucyjny	2
Wyniki AE1	3
AE2: Algorytm ewolucyjny dla cutting stock problem	4
Wyniki AE2	6
AE3: Optymalizacja wag w sieci MLP z użyciem algorytmu genetycznego	8

Implementacja Algorytmów Ewolucyjnych

AE1: podstawowy algorytm ewolucyjny

- **Cel:** Implementacja podstawowego algorytmu ewolucyjnego do znajdowania minimalnych wartości funkcji.

Algorytm ewolucyjny

W eksperymencie wykorzystano klasyczny algorytm ewolucyjny do minimalizacji funkcji celu. Osobniki w populacji reprezentowane są jako wektory rzeczywistego wymiaru R^d , gdzie d to liczba wymiarów przestrzeni poszukiwań.

Parametry algorytmu

- Rozmiar populacji: N
- Współczynnik mutacji: μ
- Odchylenie standardowe mutacji: σ
- Liczba pokoleń: G
- Przedział ograniczeń: $[a, b]$

Opis algorytmu

Algorytm przebiega według następujących kroków:

1. **Inicjalizacja populacji:** Wygenerowano N osobników losowo z przedziału $[a, b]^d$:

$$\mathbf{x}_i \sim \mathcal{U}(a, b)^d, \quad i = 1, \dots, N$$

2. **Ocena populacji:** Dla każdego osobnika obliczana jest wartość funkcji celu $f(\mathbf{x}_i)$.
3. **Selekcja:** Do krzyżowania wybierani są rodzice metodą turniejową z losowej próby $k = 3$ osobników, z których wybierany jest ten o najlepszej wartości funkcji celu.
4. **Krzyżowanie (one-point crossover):** Dla dwóch rodziców $\mathbf{p}_1, \mathbf{p}_2 \in R^d$ losowany jest punkt przecięcia $c \in \{1, \dots, d-1\}$, a następnie tworzona jest para dzieci:

$$\mathbf{c}_1 = (\mathbf{p}_1^{[:c]}, \mathbf{p}_2^{[c:]})$$

$$\mathbf{c}_2 = (\mathbf{p}_2^{[:c]}, \mathbf{p}_1^{[c:]})$$

5. **Mutacja:** Każdy gen potomka jest z prawdopodobieństwem μ modyfikowany przez dodanie szumu Gaussowskiego:

$$x_i \leftarrow x_i + \mathcal{N}(0, \sigma^2)$$

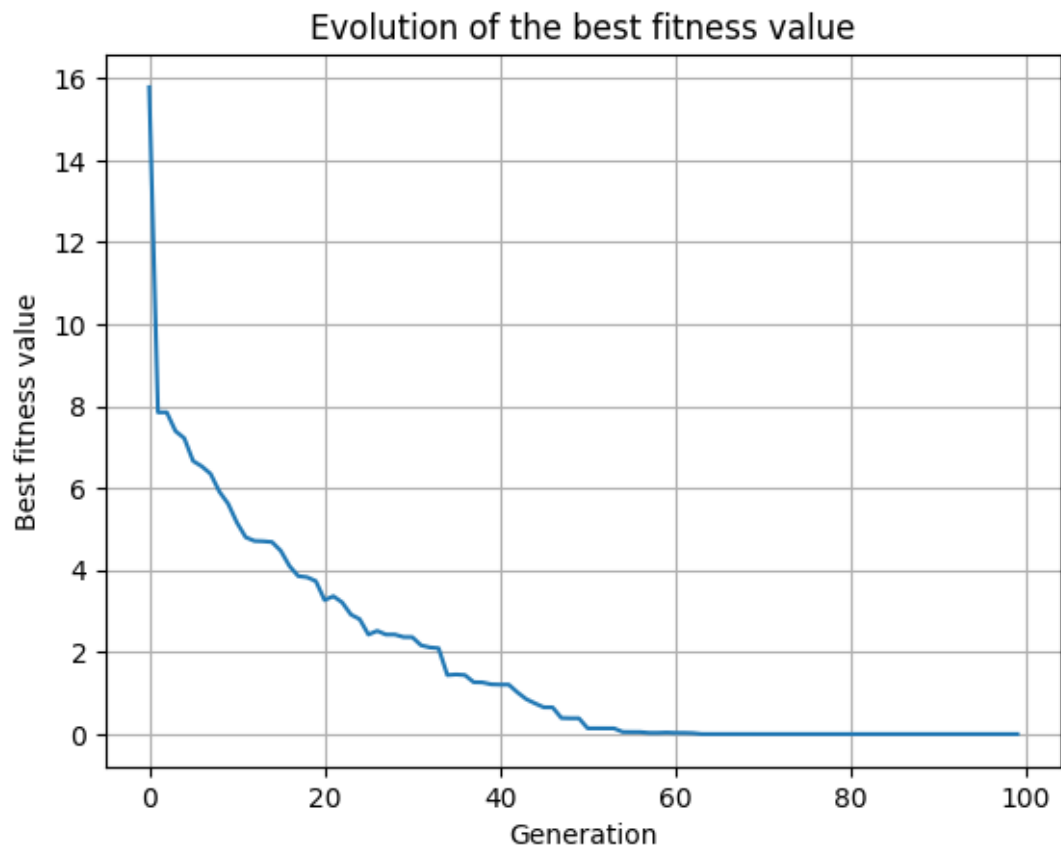
6. **Nowa generacja:** Powtarzając selekcję, krzyżowanie i mutację, tworzy się nową populację o tej samej liczebności. Proces ten powtarzany jest przez G pokoleń.
7. **Zapamiętywanie najlepszego rozwiązania:** W każdej generacji śledzony jest najlepszy osobnik, a jego wartość funkcji celu zapisywana jest w historii.

Zwracane wartości

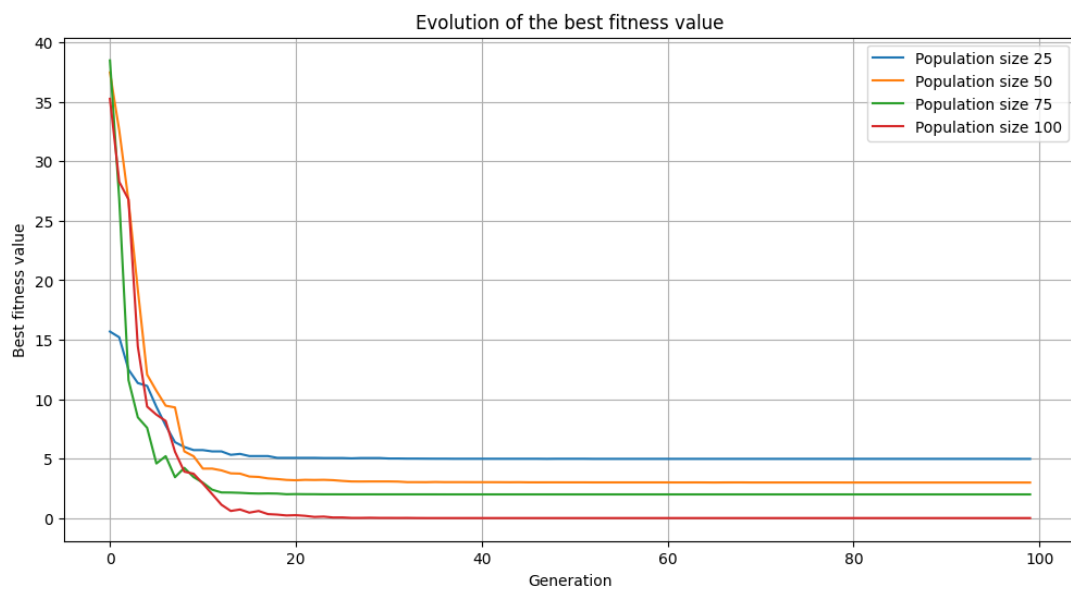
Po zakończeniu algorytmu zwracane są:

- Najlepszy znaleziony osobnik $\mathbf{x}^*$
- Wartość funkcji celu $f(\mathbf{x}^*)$
- Historia najlepszych wartości funkcji celu w kolejnych pokoleniach

Wyniki AE1



Rys. 1. Ewolucja algorytmu dla funkcji trójwymiarowej (populacja=25).



Rys. 2. Ewolucja algorytmu dla funkcji Rastrigina dla różnych wielkości populacji.

AE2: Algorytm ewolucyjny dla cutting stock problem

Algorytm ewolucyjny został zaadaptowany do rozwiązania wariantu problemu cięcia (Cutting Stock Problem), polegającego na maksymalizacji łącznej wartości prostokątów umieszczonych wewnątrz koła o zadanym promieniu. Każdy prostokąt charakteryzowany jest przez trójkę (w, h, v) , oznaczającą odpowiednio szerokość, wysokość oraz wartość.

Reprezentacja osobników

Każdy osobnik reprezentuje listę identyfikatorów prostokątów, które są następnie deterministycznie układane w kole w sposób przypominający siatkę. Każdy prostokąt jest przymierzany od lewej górnej części koła, schodząc w dół i przesuwając się w prawo po zajęciu kolumny. Dla każdej konfiguracji wyznaczana jest pozycja środka prostokąta (x_c, y_c) .

Ocena przystosowania

Funkcja celu (fitness) jest zdefiniowana jako suma wartości v wszystkich prostokątów, które:

- mieszczą się w całości w obrębie koła (wszystkie cztery rogi muszą spełniać nierówność $x^2 + y^2 \leq R^2$),
- nie zachodzą na inne wcześniej rozmieszczone prostokąty.

Inicjalizacja populacji

Populacja początkowa jest generowana losowo poprzez wielokrotne wywołanie procedury rozmieszczania prostokątów. Dla każdego osobnika generowana jest lista identyfikatorów prostokątów oraz odpowiadająca jej konfiguracja przestrzenna.

Selekcja turniejowa

Selekcja turniejowa o rozmiarze k polega na losowym wyborze k osobników z populacji i wybraniu tego z nich, który osiąga najwyższą wartość funkcji celu.

Krzyżowanie

Krzyżowanie dwóch rodziców polega na stworzeniu nowego osobnika poprzez wybranie identyfikatorów prostokątów naprzemiennie z obydwu list rodziców. Długość nowego osobnika odpowiada długości krótszego z rodziców.

Mutacja

Z prawdopodobieństwem p_{mut} jeden z identyfikatorów w nowym osobniku zostaje zastąpiony losowym identyfikatorem prostokąta z oryginalnego zbioru. Mutacja ma na celu zwiększenie różnorodności populacji.

Ewolucja

Algorytm iteruje przez G pokoleń. W każdej generacji:

1. tworzona jest nowa populacja poprzez selekcję, krzyżowanie i mutację,

2. najlepszy osobnik aktualizuje globalne optimum, jeśli jego wartość celu jest wyższa niż dotychczas najlepsza,
3. zapisywana jest historia najlepszej wartości funkcji celu dla wizualizacji postępu algorytmu.

Wizualizacja wyników

Dla najlepszego osobnika dostępna jest wizualizacja rozmieszczenia prostokątów w obrębie koła. Prostokąty poprawnie umieszczone oznaczone są kolorem zielonym, a te wychodzące poza obszar lub nachodzące na inne – czerwonym. Dodatkowo tworzony jest wykres przebiegu wartości funkcji celu w czasie.

Parametry

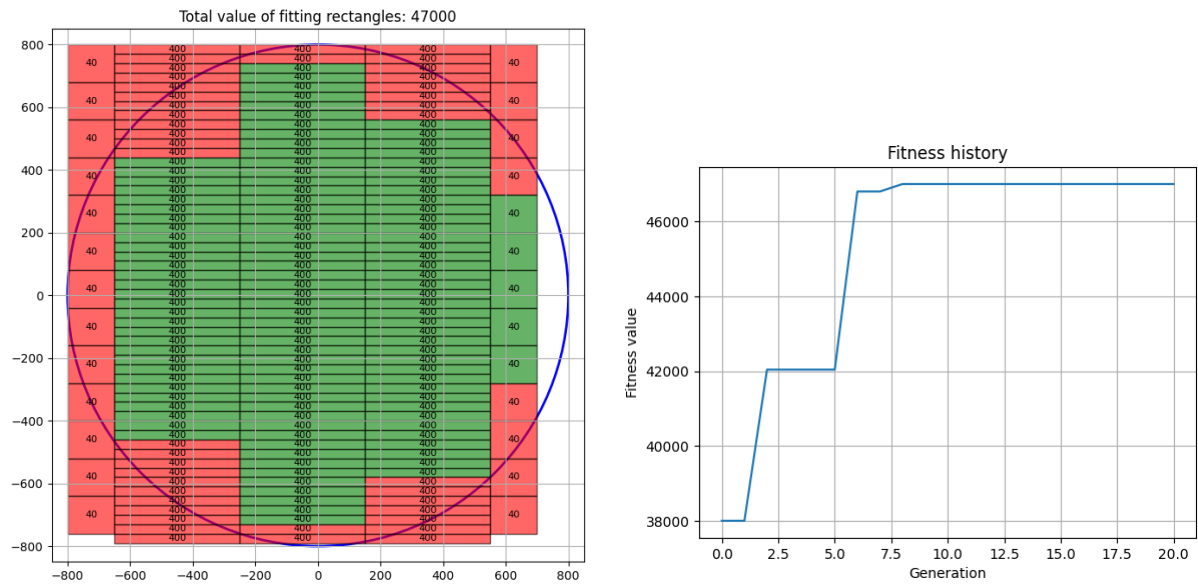
- Populacja: $N = 100$
- Liczba pokoleń: $G = 10$
- Prawdopodobieństwo mutacji: $p_{mut} = 0.1$
- Rozmiar turnieju: $k = 3$
- Promień koła: R
- Zbiór prostokątów: $\{(w_i, h_i, v_i)\}_{i=1}^n$

Cel optymalizacji

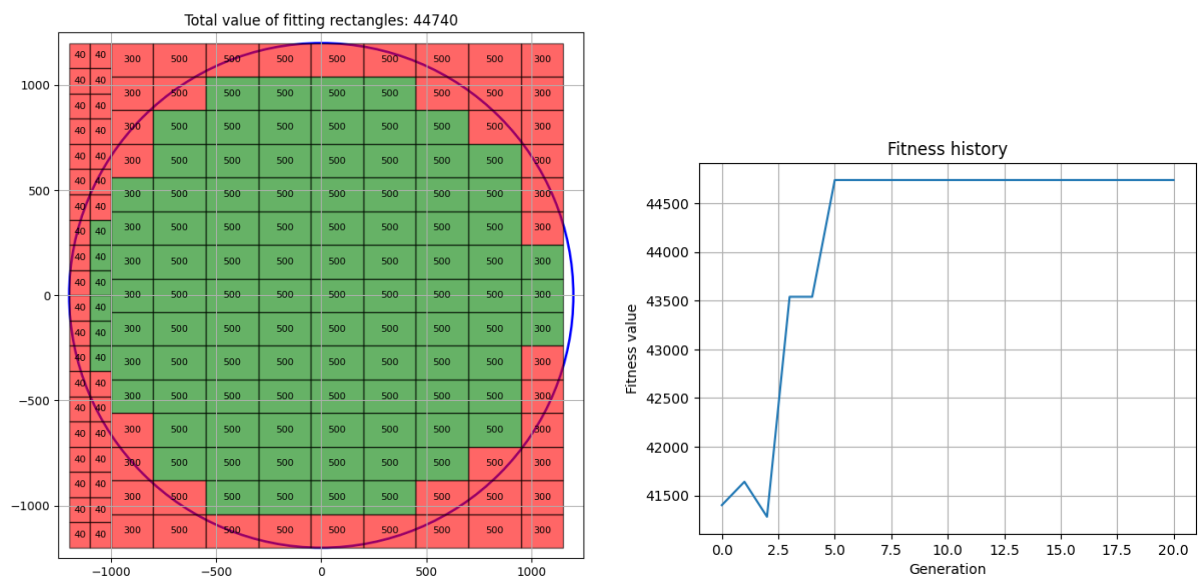
$$\max_{\text{układ}} \sum_{i=1}^m v_i \cdot \mathbf{1}_{\text{pasuje}}(i) \quad (1)$$

gdzie v_i to wartość i -tego prostokąta, a $\mathbf{1}_{\text{pasuje}}(i)$ to funkcja wskaźnikowa przyjmująca wartość 1 tylko jeśli prostokąt mieści się w kole i nie koliduje z innymi.

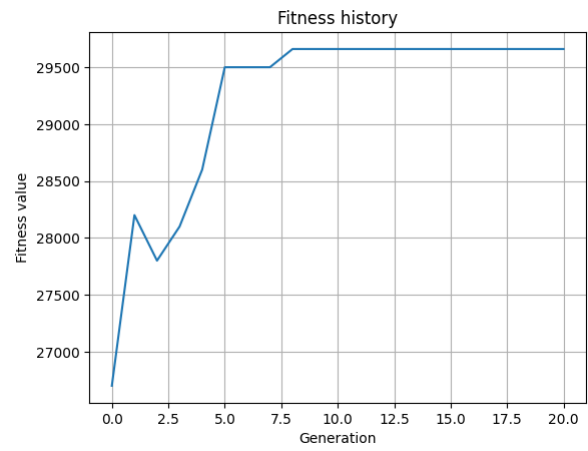
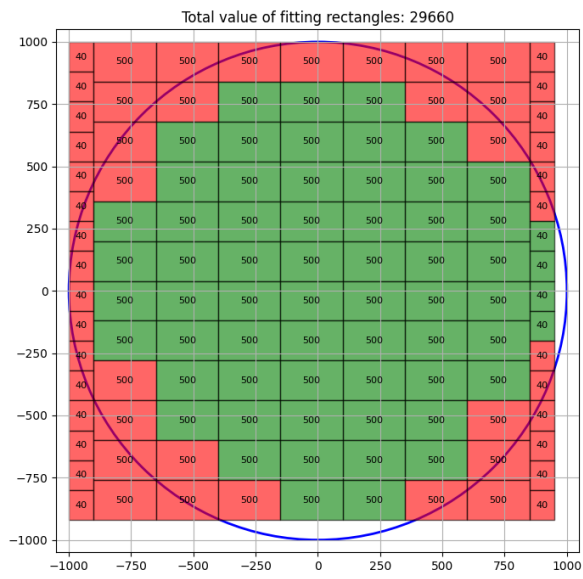
Wyniki AE2



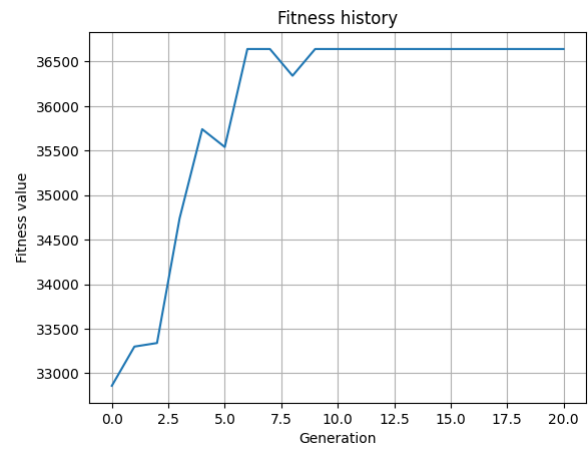
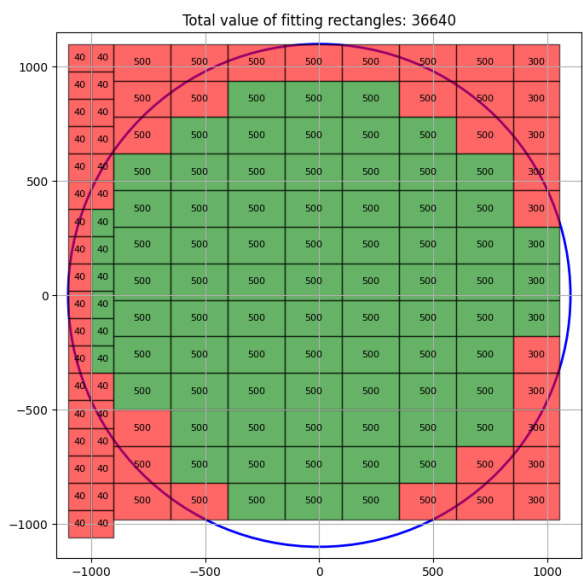
Rys. 3. Radius 800 circle (acceptance threshold: 30000).



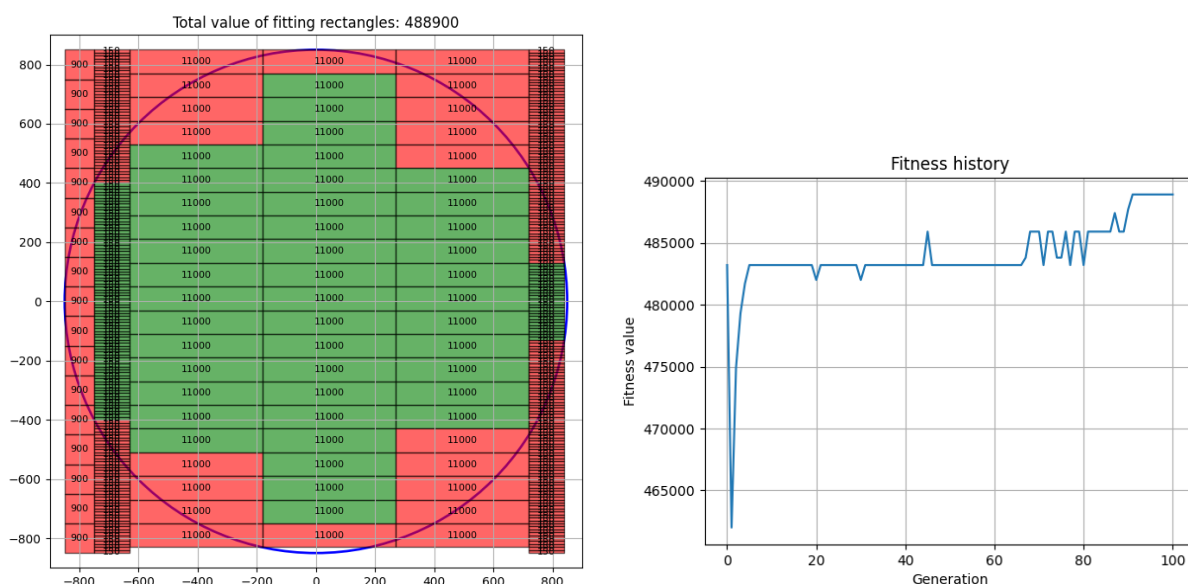
Rys. 4. Radius 1200 circle (acceptance threshold: 30000).



Rys. 5. Radius 1000 circle (acceptance threshold: 17500).



Rys. 6. Radius 1100 circle (acceptance threshold: 25000).



Rys. 7. Radius 850 circle.

AE3: Optymalizacja wag w sieci MLP z użyciem algorytmu genetycznego

W ramach zadania AE3 zaimplementowałem algorytm genetyczny do uczenia sieci MLP, w której optymalizowany był wektor wag poprzez minimalizację błędu na zbiorze uczącym (funkcja kosztu: MSE). Algorytm przyjmował jako dane wejściowe strukturę sieci (liczbę neuronów w warstwach) oraz dane treningowe. Kluczowymi elementami rozwiązania były:

- **Reprezentacja chromosomu** – wektor zawierający spłaszczone wagi i biasy sieci MLP.
- **Funkcja fitness** – odwrotność błędu MSE na zbiorze uczącym.
- **Operatory GA:**
 - *Krzyżowanie (crossover)*: testowałem różne warianty, m.in. jednopunktowe, dwupunktowe oraz uśrednianie wag potomków.
 - *Mutacja*: losowa zmiana wartości jednej lub kilku wag w chromosomie (np. przez dodanie szumu gaussowskiego).
 - *Selekcja*: zastosowałem selekcję turniejową oraz ruletkową.

Uczenie zostało przeprowadzone dla trzech zbiorów danych:

- **Iris** – klasyfikacja 3 klas kwiatów,
- **Multimodal-large** – zbiór z laboratoriów o sieciach neuronowych,
- **Auto-MPG** – regresja, przewidywanie wartości MPG.

Wyniki i obserwacje

Pomimo testowania różnych wariantów operatorów genetycznych i parametrów (rozmiar populacji, współczynniki mutacji, liczba pokoleń), algorytm nie uzyskiwał zadowalających wyników na żadnym ze zbiorów. Główne problemy, które zaobserwowałem:

- **Wpadanie w minima lokalne** – populacja szybko traciła różnorodność, a rozwój fitnessu zatrzymywał się po kilku pokoleniach.

- **Brak stabilnej konwergencji** – jakość rozwiązań była niestabilna, mocno zależna od losowej inicjalizacji.
- **Trudności w zbalansowaniu eksploracji i eksploatacji** – zbyt mała presja selekcyjna lub nadmierna eksploatacja prowadziły do stagnacji.

Podsumowując, choć algorytm został zaimplementowany zgodnie z wymaganiami, nie udało się osiągnąć satysfakcjonujących wyników predykcyjnych. W przyszłości możliwe usprawnienia to m.in. zastosowanie elityzmu, adaptacyjnej mutacji lub hybrydyzacja z metodami gradientowymi.